

Notebook 2.1 Supervised Learning

The purpose of this notebook is to explore the linear regression model discussed in Chapter 2 of the book.

Work through the cells below, running each cell in turn. In various places you will see the words "TODO". Follow the instructions at these places and write code to complete the functions. There are also questions interspersed in the text.

Contact me at udlbookmail@gmail.com if you find any mistakes or have any suggestions.

```
In [1]: # Math Library
import numpy as np
# Plotting Library
import matplotlib.pyplot as plt
```

```
In [2]: # Create some input / output data
x = np.array([0.03, 0.19, 0.34, 0.46, 0.78, 0.81, 1.08, 1.18, 1.39, 1.60, 1.65, 1.9
y = np.array([0.67, 0.85, 1.05, 1.0, 1.40, 1.5, 1.3, 1.54, 1.55, 1.68, 1.73, 1.6 ])

print(x)
print(y)

[0.03 0.19 0.34 0.46 0.78 0.81 1.08 1.18 1.39 1.6  1.65 1.9 ]
[0.67 0.85 1.05 1.  1.4  1.5  1.3  1.54 1.55 1.68 1.73 1.6 ]
```

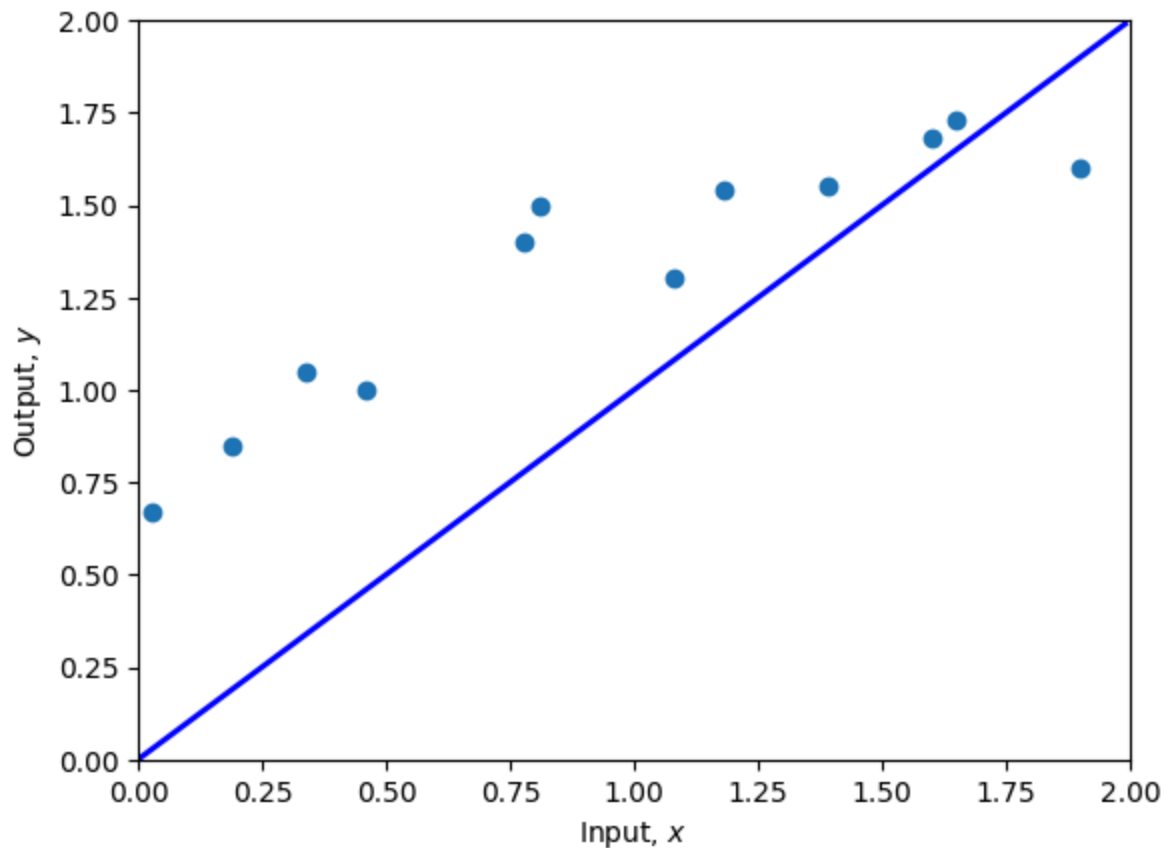
```
In [3]: # Define 1D Linear regression model
def f(x, phi0, phi1):
    # TODO : Replace this line with the linear regression model (eq 2.4)
    y = x

    return y
```

```
In [4]: # Function to help plot the data
def plot(x, y, phi0, phi1):
    fig, ax = plt.subplots()
    ax.scatter(x, y)
    plt.xlim([0, 2.0])
    plt.ylim([0, 2.0])
    ax.set_xlabel('Input, $x$')
    ax.set_ylabel('Output, $y$')
    # Draw Line
    x_line = np.arange(0, 2, 0.01)
    y_line = f(x_line, phi0, phi1)
    plt.plot(x_line, y_line, 'b-', lw=2)

    plt.show()
```

```
In [5]: # Set the intercept and slope as in figure 2.2b
phi0 = 0.4 ; phi1 = 0.2
# Plot the data and the model
plot(x,y,phi0,phi1)
```



```
In [6]: # Function to calculate the loss
def compute_loss(x,y,phi0,phi1):

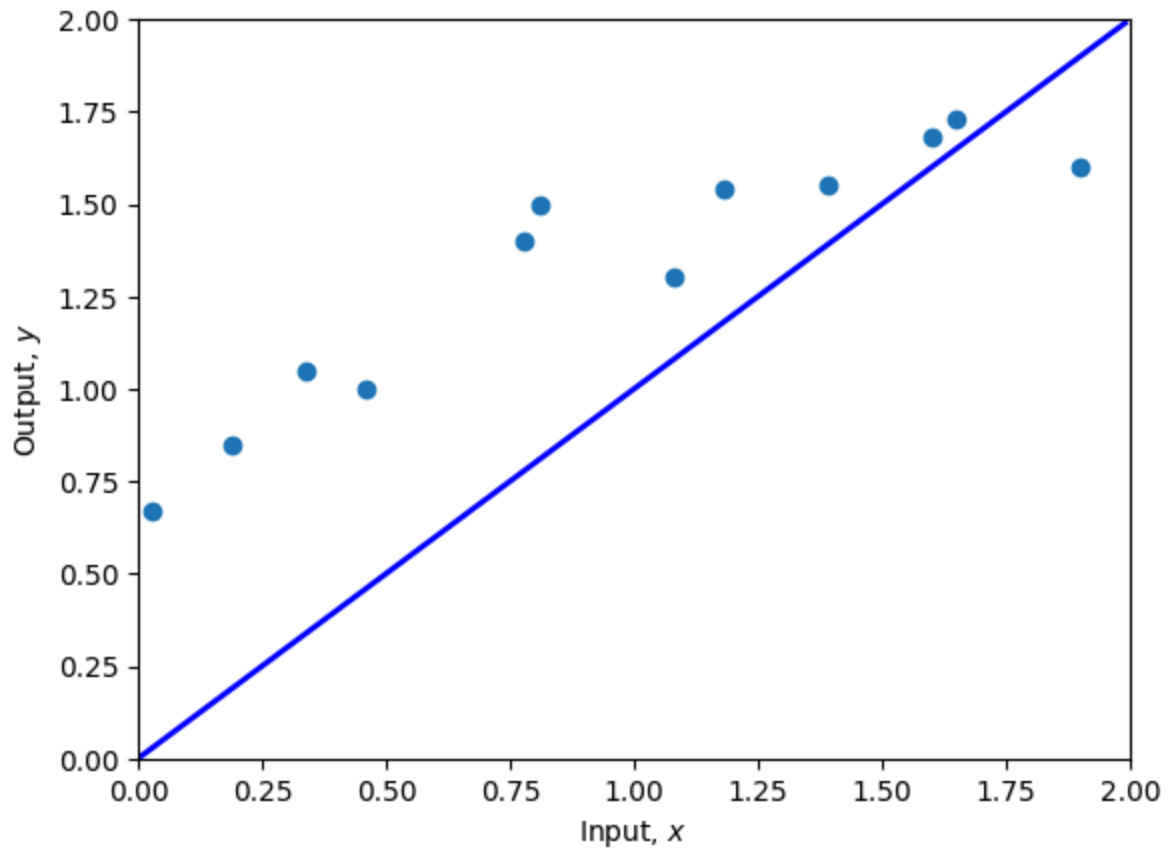
    # TODO Replace this line with the loss calculation (equation 2.5)
    loss = 0

    return loss
```

```
In [7]: # Compute the Loss for our current model
loss = compute_loss(x,y,phi0,phi1)
print(f'Your Loss = {loss:3.2f}, Ground truth =7.07')
```

Your Loss = 0.00, Ground truth =7.07

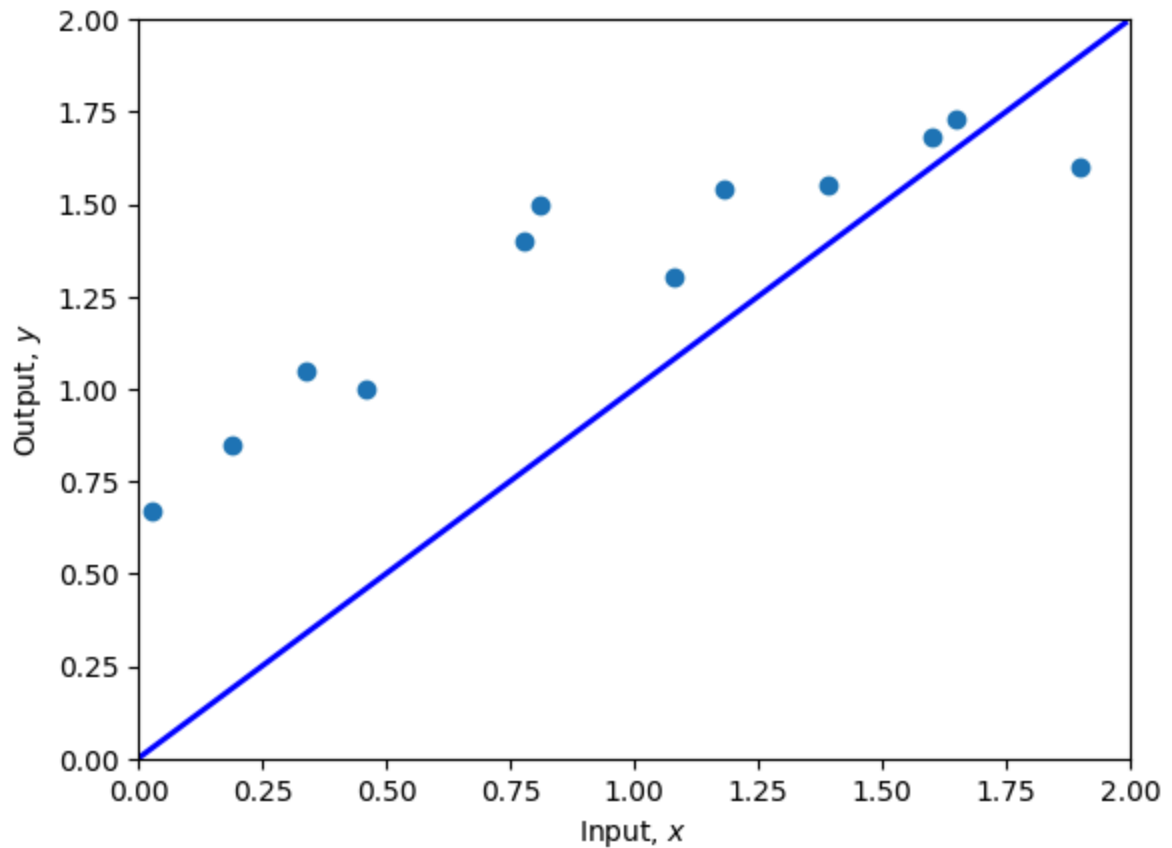
```
In [8]: # Set the intercept and slope as in figure 2.2c
phi0 = 1.60 ; phi1 = -0.8
# Plot the data and the model
plot(x,y,phi0,phi1)
loss = compute_loss(x,y,phi0,phi1)
print(f'Your Loss = {loss:3.2f}, Ground truth =10.28')
```



Your Loss = 0.00, Ground truth =10.28

```
In [9]: # TODO -- Change the parameters manually to fit the model
# First fix phi1 and try changing phi0 until you can't make the loss go down any mo
# Then fix phi0 and try changing phi1 until you can't make the loss go down any mor
# Repeat this process until you find a set of parameters that fit the model as in f
# You can either do this by hand, or if you want to get fancy, write code to descen
# Start at these values:
phi0 = 1.60 ; phi1 = -0.8

plot(x,y,phi0,phi1)
print(f'Your Loss = {compute_loss(x,y,phi0,phi1):3.2f}')
```



Your Loss = 0.00

Visualizing the loss function

The above process is equivalent to descending coordinate wise on the loss function

Now let's plot that function

```
In [10]: # Make a 2D grid of possible phi0 and phi1 values
phi0_mesh, phi1_mesh = np.meshgrid(np.arange(0.0,2.0,0.02), np.arange(-1.0,1.0,0.02)

# Make a 2D array for the losses
all_losses = np.zeros_like(phi1_mesh)
# Run through each 2D combination of phi0, phi1 and compute loss
for indices,temp in np.ndenumerate(phi1_mesh):
    all_losses[indices] = compute_loss(x,y, phi0_mesh[indices], phi1_mesh[indices])
```

```
In [11]: # Plot the loss function as a heatmap
fig = plt.figure()
ax = plt.axes()
fig.set_size_inches(7,7)
levels = 256
ax.contourf(phi0_mesh, phi1_mesh, all_losses ,levels)
levels = 40
ax.contour(phi0_mesh, phi1_mesh, all_losses ,levels, colors=['#80808080'])
ax.set_ylim([1,-1])
ax.set_xlabel(r'Intercept, $\phi_0$')
```

```
ax.set_ylabel(r'Slope,  $\phi_1$ ')
```

```
# Plot the position of your best fitting line on the loss function
```

```
# It should be close to the minimum
```

```
ax.plot(phi0,phi1,'ro')
```

```
plt.show()
```

